Neural Forge Final Report

Merrick Shorter, Alyx Snyder, Karmen Whitson, Antonio Johnson

Mathematics, Computer & Natural Sciences Division, Ohio Dominican University

CIS 234: Database Management Systems

Dr. Alae Loukili

December 10, 2025

Table of Contents

Introduction 3

High-Level Project Description 3

Analysis Phase 3

- Purpose behind the database 3
- Client Requirements 4
- Neural Forge Scenario 4
- Entity Relationship Diagram 5

Design Phase 6

- Database Diagram 6
- Data Dictionary 6
- Normalization 7
- Relationship Constraints 7
- Schema 8

Implementation Phase 8

Lessons Learned 8

Appendix 9

- Table Creation Scripts 9
  - Sites table 9
  - Chips table 10
  - Assembly_lines table 10
  - Production_records table 11
  - Downtime_events table 11
- Data Insertion 12
  - Site data 12
  - Chip data 12
  - Assembly_line data 12
- Data Script for production_record and downtime_event 13
- Individual SQL Queries 20
- Application Program 22

Introduction

Code Works LLC was contracted by Neural Forge, the chip manufacturing company, to create a reliable database and some reports connected to the database. The purpose behind this database is to track the productivity of the 12 manufacturing sites in the United States owned by Neural Forge. From the reports, high-level executives can see proven data pointing to weak points in their company on an assembly line level, ensuring quotas are met and any issues can be resolved. Code Works LLC is familiar with this kind of job as they have helped countless companies in a wide range of domains track their data for increased company-wide productivity.

High-Level Project Description

Code Works gathered requirements for the contracted database then began developing a data model to show how entities would behave and relationships would look like between these entities. After that entities were given attributes and eventually keys. Eventually in the implementation phase a schema and tables were created using Entity Framework Core which automatically handles DOS attacks and is an industry standard that makes future migrations simple and handles a lot of behind the scenes issues with databases. In order to get the data from the machines to the database Code Works created a C# backend with api endpoints for machines on the assembly line to pass data to hourly. Finally, data scientists at Code Works set up a few PowerBI reports based on important metrics the team had determined during requirement gathering talks with Neural Forge executives. The collection of the work done on this project allows Nerual Forge to find pain points in the manufacturing process and show this data to executives and shareholders to communicate what is happening across the country in Neural Forge factories.

Analysis Phase

Purpose behind the database

Quantify the productivity of each manufacturing plant and track data to ensure we meet quotas and gain the ability to explain why productivity may be down due to outages, maintenance, etc. We can further use this data to show company executives and shareholders, giving the company better transparency and understanding of the manufacturing process.

Client Requirements

Relational Database

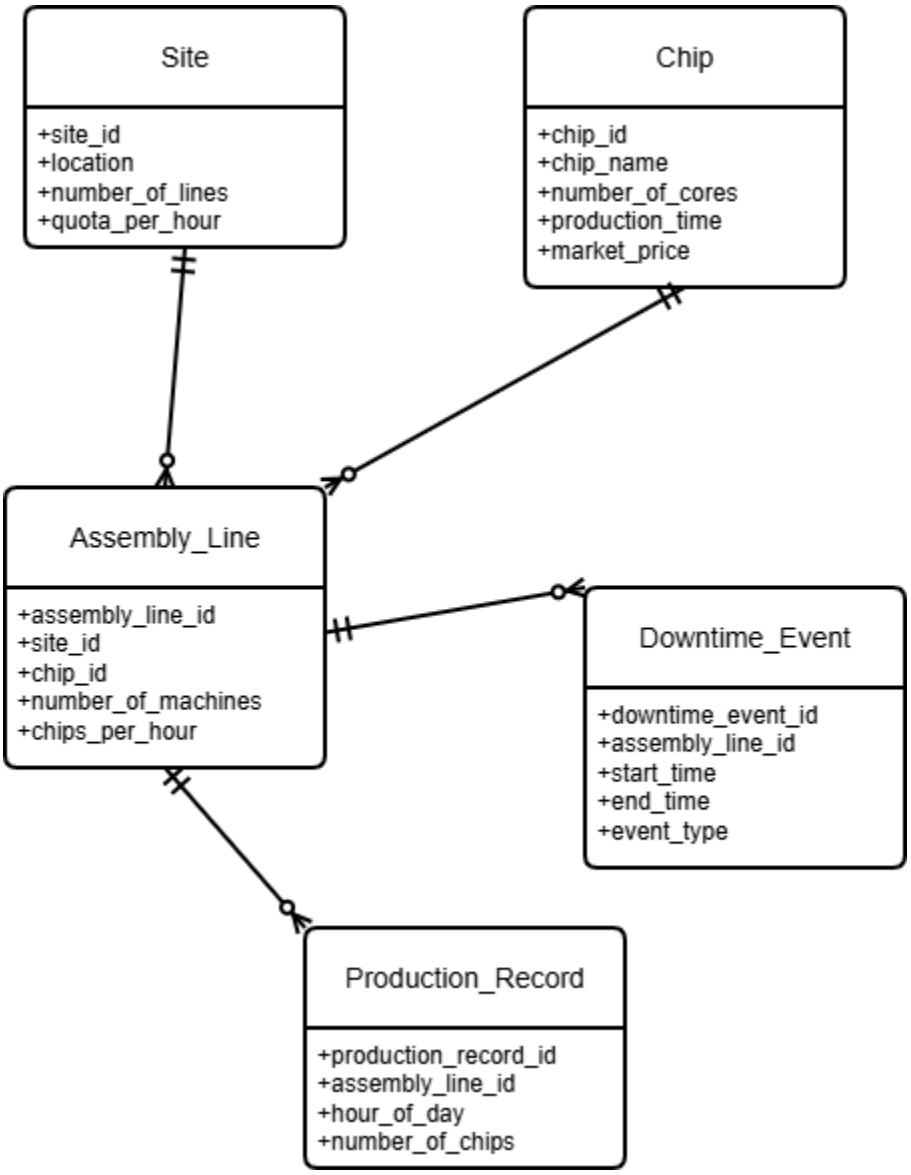
Data pipeline from input → Api → database → visualization

Clean, structured visualizations to show company executives and shareholders.

Neural Forge Scenario

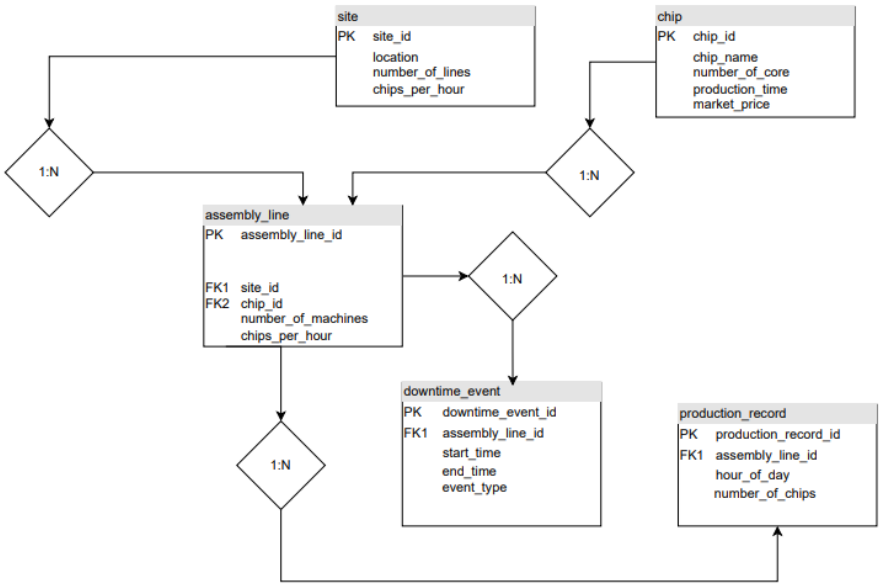
This database is for a chip manufacturing company with many regional manufacturing plants. We will keep track of the productivity of each site. For each site, we will keep track of each assembly line (by number) and their quota (chips made per hour). For each assembly line, we are keeping track of the chip it is making, how many machines are needed for the line, and the quota (per hour). For this, we need to keep track of information for chips. Including the name, number of cores, production time, and market price. The dynamic parts of our database will be production records, which track the number of chips per hour per site, and the downtime events, which keep track of when assembly lines go down and for what reason.

Entity Relationship Diagram



Design Phase

Database Diagram



Data Dictionary

site

Column Name	Data Type	Key	Required	Default
site_id	Int	Primary	Yes	Auto Increment
location	VarChar(100)	No	Yes	None
number_of_lines	Int	No	No	Null
chips_per_hour	Int	No	No	Null

chip

Column Name	Data Type	Key	Required	Default
chip_id	Int	Primary	Yes	Auto Increment
chip_name	VarChar(100)	No	Yes	None
number_of_core	Int	No	No	Null
production_time	Time	No	No	Null
market_price	Numeric(10,2)	No	No	Null

assembly_line

Column Name	Data Type	Key	Required	Default
assembly_line_id	Int	Primary	Yes	Auto Increment
site_id	Int	Foreign	Yes	None
chip_id	Int	Foreign	Yes	None
number_of_machines	Int	No	No	Null
chips_per_hour	Int	No	No	Null

production_record

Column Name	Data Type	Key	Required	Default
production_record_id	Int	Primary	Yes	Auto Increment
assembly_line_id	Int	Foreign	Yes	None
hour_of_day	DateTime	No	Yes	None
number_of_chips	Int	No	No	None

downtime_event

Column Name	Data Type	Key	Required	Default
downtime_event_id	Int	Primary	Yes	Auto Increment
assembly_line_id	Int	Foreign	Yes	None
start_time	DateTime	No	Yes	None
end_time	DateTime	No	No	Null
event_type	VarChar(50)	No	No	None

Normalization

Our database design is in the 3rd normal form because every record in each table is uniquely identifiable, is atomic, and has a primary key (first normal form), all the non-key attributes depend on the entire primary key, and we have separated data that may have been duplicated (second normal form), and lastly, there are no transitive dependencies and relationships between tables are handled between foreign keys. Additionally, our design satisfies the requirements of Boyce-Codd normal form, where all primary keys are surrogate keys that are super-keys for functional dependencies

Relationship Constraints

Parent	Child	Referential Integrity Constraint	Cascading Behavior On Update/Delete
Site	assembly_line	site_id in assembly_line must exist in site	YES/YES
Chip	assembly_line	chip_id in assembly_line must exist in chip	NO/YES
assembly_line	downtime_event	assembly_line_id in downtime_event must exist in assembly_line	NO/NO
assembly_line	production_record	assembly_line_id in	NO/NO

production_record must exist in assembly_line

Schema

SITE(SiteID, Location, NumberOfLines, ChipsPerHour)

CHIP(ChipID, ChipName, NumberOfCores,ProductionTime, MarketPrice)

ASSEMBLYLINE(AssemblyLineID, *SiteID*, ChipID, NumberOfMachines, ChipsPerHour)

DOWNTIMEEVENT(*AssemblyLineID*, StartTime, EndTime, EventType)

PRODUCTIONRECORD(*AssemblyLineID*, HourOfDay, NumberOfChips)

Implementation Phase

Table Creation: See Appendix Table Creation Scripts

Data Insertion: See Appendix Data Insertion Scripts

Mock Data Script: See Appendix Mock Data Scripts

7 Individual Queries: See Appendix Individual Queries

Lessons Learned

Throughout this process I have learned a lot. I acted as the project manager for this project which is a rule I assumed by default because nobody else wanted to and I believed could best tie together everyone's skills in our group. This role reaffirmed that I do not want to be a project manager at this point in my career. I am an ambitious person, and I felt like none of the other group members matched my enthusiasm which made it hard to get excited about collaboration. With the attempted use of version control for this project I learned how to properly set up repository rules, workflow files, and proper documentation via .md files to help guide those working alongside me. On top of this knowledge, I found out the hard way that not all operating systems are created equally, and the slight changes can throw a wrench into any documented step by step guides. The request for proposal and proposal part of the project was underappreciated in my opinion. This was a simple way to show how companies/individuals handle contracted work. If you are looking for feedback on this part of the project, might I suggest you give each group a request for proposal which

while limiting creativity, this clears up a lot of confusion that every group had at the beginning of the project. This analysis phase opened my eyes to requirement gathering which I see now is very important and one of the things that could've improved the fluidity of my summer internship. Learning to take it one step at a time with the Analysis phase took a lot of patience for me but I now understand the importance of this because if one party involved does not understand exactly the goals and methods of the project people get lost. The design phase is when it started to feel like we were going to have a final product. Here communication was the most important skill to ensure every member working on the different diagrams agreed on and understood the ER diagram. Communication is very important in tech teams, especially if people on your team are remote. Finally, we got to the implementation phase which went smoothly thanks to the diligence we had during the rest of the steps. I spent this time helping each of my group members get a local copy of the database on their computer using EF Core which was a great exercise for the real world when you onboard people. Overall, I believe this project is a cornerstone of the Ohio Dominican Computer Science course work and could be one of the more important projects of many of the students resumes. To solidify that there should be more emphasis on it as something that could get you hired in the future. If possible, the application part at the end should be flushed out more to show that you not only know how to create a database schema but also a application that can use it.

Appendix

Table Creation Scripts

Sites table

```
CREATE TABLE `sites` (
  `site_id` int NOT NULL AUTO_INCREMENT,
  `location` varchar(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT NULL,
  `number_of_lines` int NOT NULL,
  `chips_per_hour` int NOT NULL,
  PRIMARY KEY (`site_id`)
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Chips table

```
CREATE TABLE `chips` (
  `chip_id` int NOT NULL AUTO_INCREMENT,
  `chip_name` varchar(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT NULL,
  `number_of_cores` int NOT NULL,
  `production_time` double NOT NULL,
  `market_price` double NOT NULL,
  PRIMARY KEY (`chip_id`)
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Assembly_lines table

```
CREATE TABLE `assembly_lines` (
  `assembly_line_id` int NOT NULL AUTO_INCREMENT,
  `site_id` int NOT NULL,
  `chip_id` int NOT NULL,
  `number_of_machines` int DEFAULT NULL,
  `chips_per_hour` int NOT NULL,
  PRIMARY KEY (`assembly_line_id`),
  KEY `IX_assembly_lines_chip_id` (`chip_id`),
  KEY `IX_assembly_lines_site_id` (`site_id`),
  CONSTRAINT `FK_assembly_lines_chips_chip_id` FOREIGN KEY (`chip_id`) REFERENCES `chips` (`chip_id`) ON DELETE RESTRICT,
  CONSTRAINT `FK_assembly_lines_sites_site_id` FOREIGN KEY (`site_id`) REFERENCES `sites` (`site_id`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Production_records table

```
CREATE TABLE `production_records` (
  `production_record_id` int NOT NULL AUTO_INCREMENT,
  `assembly_line_id` int NOT NULL,
  `number_of_chips` int NOT NULL,
  `hour_of_day` datetime(6) NOT NULL,
  PRIMARY KEY (`production_record_id`),
  KEY `IX_production_records_assembly_line_id` (`assembly_line_id`),
  CONSTRAINT `FK_production_records_assembly_lines_assembly_line_id` FOREIGN KEY (`assembly_line_id`) REFERENCES `assembly_lines` (`assembly_line_id`)
) ENGINE=InnoDB AUTO_INCREMENT=3 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Downtime_events table

```
CREATE TABLE `downtime_events` (
  `downtime_event_id` int NOT NULL AUTO_INCREMENT,
  `assembly_line_id` int NOT NULL,
  `start_time` datetime(6) NOT NULL,
  `end_time` datetime(6) NOT NULL,
  `event_type` longtext CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci,
  PRIMARY KEY (`downtime_event_id`),
  KEY `IX_downtime_events_assembly_line_id` (`assembly_line_id`),
  CONSTRAINT `FK_downtime_events_assembly_lines_assembly_line_id` FOREIGN KEY (`assembly_line_id`) REFERENCES `assembly_lines` (`assembly_line_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

Data Insertion

Site data

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('New Albany, OH', '8', '400');

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('Arona, PA', '6', '300');

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('Raymond City, WV', '6', '300');

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('Mt Sterling, KY', '6', '300');

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('Bedford, IN', '6' , '300');

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('Thomasboro, IL', '6', '300');

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('Portage, MI', '8', '400');

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('Chipewa Falls, WI', '6', '300');

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('Appleton, WI', '6', '300');

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('Paynesville, MN', '4', '300');

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('Canova, SD', '4', '200');

Insert into neural_forge_prod.sites (location, number_of_lines, chips_per_hour) Values ('Lincoln, ND', '4', '200');

Chip data

Insert into neural_forge_prod.chips (chip_name, number_of_cores, production_time, market_price) Values ('NF.Core.i5.processor.14501E', '8960', '5.9', '743.99');

Insert into neural_forge_prod.chips (chip_name, number_of_cores, production_time, market_price) Values ('NF.Core.i5.processor.14401E', '6144', '3.7', '685.99');

Insert into neural_forge_prod.chips (chip_name, number_of_cores, production_time, market_price) Values ('NF.Core.i7.processor.14701E', '3840', '2.3', '519.99');

Insert into neural_forge_prod.chips (chip_name, number_of_cores, production_time, market_price) Values ('NF.Core.i7.processor.14700F', '4608', '3.4', '557.99');

Insert into neural_forge_prod.chips (chip_name, number_of_cores, production_time, market_price) Values ('NF.Gold.processor', '1540', '2.4', '223.99');

Insert into neural_forge_prod.chips (chip_name, number_of_cores, production_time, market_price) Values ('NF.Silver.processor', '2148', '2.6', '267.99');

Insert into neural_forge_prod.chips (chip_name, number_of_cores, production_time, market_price) Values ('NF.Core.i9.processor.14901E', '10752', '6.4', '936.99');

Insert into neural_forge_prod.chips (chip_name, number_of_cores, production_time, market_price) Values ('NF.Core.i9.processor.14901KS', '21760', '8.1', '1255.99');

Insert into neural_forge_prod.chips (chip_name, number_of_cores, production_time, market_price) Values ('NF.Core.i9.processor.1400', '2560', '3.2', '494.99');

Assembly_line data

-- Lines for site 1

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour)

Values

('1', '7', '18', '45')

,('1', '7', '18', '45')

,('1', '7', '18', '45')

,('1', '7', '18', '45')

,('1', '8', '20', '55')

,('1', '8', '20', '55')

,('1', '8', '20', '55')

,('1', '8', '20', '55');

-- Lines for site 2

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour) Values ('2', '2', '15', '55')

,('2', '2', '15', '55')

,('2', '2', '15', '55')

,('2', '3', '12', '45')

,('2', '3', '12', '45')

,('2', '3', '12', '45');

-- Lines for site 3

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour) Values ('3', '2', '15', '55')

,('3', '2', '15', '55')

,('3', '2', '15', '55')

,('3', '3', '12', '45')

,('3', '3', '12', '45')

,('3', '3', '12', '45');

-- Lines for site 4

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour) Values ('4', '2', '15', '55')

,('4', '2', '15', '55')

,('4', '2', '15', '55')

,('4', '3', '12', '45')

,('4', '3', '12', '45')

,('4', '3', '12', '45');

-- Lines for site 5

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour) Values ('5', '1', '20', '45')

,('5', '1', '20', '45')

,('5', '1', '20', '45')

,('5', '4', '13', '55')

,('5', '4', '13', '55')

,('5', '4', '13', '55');

-- Lines for site 6

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour) Values ('6', '1', '20', '45')

,('6', '1', '20', '45')

,('6', '1', '20', '45')

,('6', '4', '13', '55')

,('6', '4', '13', '55')

,('6', '4', '13', '55');

-- Lines for site 7

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour) Values ('7', '7', '18', '45')

,('7', '7', '18', '45')

,('7', '7', '18', '45')

,('7', '7', '18', '45')

,('7', '8', '20', '55')

,('7', '8', '20', '55')

,('7', '8', '20', '55')

,('7', '8', '20', '55');

-- Lines for site 8

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour) Values ('8', '1', '20', '45')

,('8', '1', '20', '45')

,('8', '1', '20', '45')

,('8', '4', '13', '55')

,('8', '4', '13', '55')

,('8', '4', '13', '55');

-- Lines for site 9

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour) Values ('9', '1', '20', '45')

,('9', '1', '20', '45')

,('9', '1', '20', '45')

,('9', '4', '13', '55')

,('9', '4', '13', '55')

,('9', '4', '13', '55');

-- Lines for site 10

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour) Values ('10', '9', '10', '50')

,('10', '9', '10', '50')

,('10', '9', '10', '50')

,('10', '9', '10', '50');

-- Lines for site 11

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour) Values ('11', '5', '9', '55')

,('11', '5', '9', '55')

,('11', '6', '10', '45')

,('11', '6', '10', '45');

-- Lines for site 12

Insert into neural_forge_prod.assembly_lines (site_id, chip_id, number_of_machines, chips_per_hour) Values ('12', '5', '9', '55')

,('12', '5', '9', '55')

,('12', '6', '10', '45')

,('12', '6', '10', '45');

Data Script for production_record and downtime_event

Mock_data_script for production_record and downtime_event data also available at this link https://colab.research.google.com/drive/1knrLpjVZjQN9yMXNEzfOFdLcp8M29jLj#scrollTo=yg9gmtEtL7-o :

```
import json

import random

from datetime import datetime, timedelta


def generate_production_records(lines):

    collection = []

    current_time = datetime.now().replace(minute=0, second=0, microsecond=0)

    hours = 8
```

```
    for i in range(hours):
      for m in range(1,lines+1):
        data = {
          "assemblyLineId": m,
          "numberOfChips": random.randint(35, 60),
          "hourOfDay": (current_time + timedelta(hours=i)).isoformat()
        }
        collection.append(data)

    return collection

def generate_downtime_events(lines):
    collection = []
    for i in range(1, lines+1):
      if random.random() < .75: # 25 percent chance there is an issue
        continue
        event_type = random.randint(1,3)
        start_time = datetime.now().replace(hour=random.randint (1,23))
        end_time = start_time + timedelta(hours = random.randint(2, 5))

        data = {
          "assemblyLineId": i,
          "start_time": start_time.isoformat(),
          "end_time": end_time.isoformat(),
          "eventType": event_type
          }
```

```
        collection.append(data)

    return collection
import json
import requests
from time import sleep

def post_request_production_records(collection):
  endpoint = "http://localhost:5115/api/ProductionRecord"
  for item in collection:
    try:
      response = requests.post(
          endpoint,
          json=item
      )

      if response.status_code in [200,201]:
        print(f"Success (Status: {response.status_code})")
      else:
        print(f"Failed (Status: {response.status_code})")
      sleep(1)
    except Exception as e:
      print(f"Error: {str(e)}")

def post_request_downtime_events(collection):
  endpoint = "http://localhost:5115/api/DowntimeEvent"
```

```
  for item in collection:
   try:
     response = requests.post(
        endpoint,
        json=item
     )

     if response.status_code in [200,201]:
       print(f"Success (Status: {response.status_code})")
     else:
       print(f"Failed (Status: {response.status_code})")
     sleep(1)
   except Exception as e:
     print(f"Error: {str(e)}")
#Main method that calls the string creation and api requests
if __name__ == "__main__":
  number_of_lines = 71
  production_record_strings = generate_production_records(number_of_lines)
  downtime_event_strings = generate_downtime_events(number_of_lines)
  post_request_production_records(production_record_strings)
  post_request_downtime_events(downtime_event_strings)
```

Individual SQL Queries

```
-- Needed a quick fix for an unintended foriegn key and constraint, will go back and properly change using an EF core migration
ALTER TABLE neural_forge_prod.production_records DROP CONSTRAINT FK_production_records_chips_ChipId;
```

```
ALTER TABLE neural_forge_prod.production_records DROP COLUMN ChipId;

-- Finding all the records where we are above/below the quota for the day
-- Above
SELECT pr.assembly_line_id, pr.number_of_chips, pr.hour_of_day
FROM neural_forge_prod.production_records pr
JOIN neural_forge_prod.assembly_lines al
ON pr.assembly_line_id = al.assembly_line_id
WHERE pr.number_of_chips >= al.chips_per_hour;

-- Below
SELECT pr.assembly_line_id, pr.number_of_chips, pr.hour_of_day
FROM neural_forge_prod.production_records pr
JOIN neural_forge_prod.assembly_lines al
ON pr.assembly_line_id = al.assembly_line_id
WHERE pr.number_of_chips < al.chips_per_hour;

-- Totaling the number produced in a day
SELECT DATE(pr.hour_of_day) AS date_of_record, SUM(pr.number_of_chips) AS toal,
COUNT(*) AS number_of_records
FROM neural_forge_prod.production_records pr
GROUP BY DATE(pr.hour_of_day)
ORDER BY date_of_record;

-- Seeing for how long within the past rolling calender month the systems were down
SELECT
```

```
SUM(
TIMESTAMPDIFF( minute, GREATEST(de.start_time, NOW() - INTERVAL 30 Day),
LEAST(de.end_time,NOW()))
) AS total_downtime_minutes
 FROM neural_forge_prod.downtime_events de
 WHERE de.end_time >= NOW() - INTERVAL 30 DAY
 AND de.start_time <= NOW();


-- estimating the cost of our systems being down for the past rolling calender month


SELECT
de.assembly_line_id,
(SUM(TIMESTAMPDIFF( minute, GREATEST(de.start_time, NOW() - INTERVAL 30 Day),
LEAST(de.end_time,NOW()))))/al.chips_per_hour) * c.market_price AS estimated_lost_value
FROM neural_forge_prod.downtime_events de
JOIN neural_forge_prod.assembly_lines al
ON de.assembly_line_id = al.assembly_line_id
JOIN neural_forge_prod.chips c
ON al.chip_id = c.chip_id
WHERE de.end_time >= NOW() - INTERVAL 30 DAY
AND de.start_time <= NOW()
GROUP BY de.assembly_line_id;
```

Application Program

This is all run against our c# backend which can be viewed at this location: https://github.com/ShoM26/NeuralForge-CIS-234

A demonstration can be seen at this link, and as promised, our PowerBI report is kinked below. Neural Forge Power BI Reports.pdf (Command Line)